

# Electro Pi — AI Engineer Technical Test

Practical / Hands-on Assessment — Mid-Level (3+ yrs)

|                        |                                                                          |
|------------------------|--------------------------------------------------------------------------|
| <b>Role</b>            | AI Engineer — Mid Level (3 yrs experience)                               |
| <b>Location</b>        | Maadi, Cairo                                                             |
| <b>Time Limit</b>      | 5–7 days from receipt (open-book, take-home)                             |
| <b>Skills Assessed</b> | LiveKit · LLMs · LangChain · Quantization · Model Deployment             |
| <b>Submission</b>      | GitHub repo (or zip) with code, README, and a short write-up per section |

## Overview

This is a hands-on practical test, not a quiz. Each section maps to a required skill from the job ad and asks you to build something small but real. Partial, working solutions are valued over incomplete, over-engineered ones. Use any language/framework version you're comfortable with, but Python 3.10+ is expected for most sections. Comment your trade-offs — we care as much about your reasoning as your code.

## Section 1 — LiveKit Agents (Real-time Voice AI)

### Task 1.1: Build a minimal voice agent with a tool call

التقدير: 20 نقطة

Using the livekit-agents Python SDK, build a minimal voice agent that:

- Runs an AgentSession pipeline: STT → LLM → TTS (any providers you have API keys for, or stub them — e.g. mock STT/TTS with text I/O if you don't have provider keys, as long as the LLM + tool-calling logic is real).
- Defines an Agent subclass with a system persona (e.g. "You are a support assistant for a food delivery app").
- Exposes at least one `@function_tool`-decorated method the LLM can call mid-conversation — e.g. `get_order_status(order_id: str) -> str` returning a mocked lookup result.
- Demonstrates, via a transcript/log or short screen recording, the LLM actually invoking the tool call during a live or simulated session.

Write-up (half page): explain how you would extend this to support barge-in / interruption handling, and how you'd add a second tool safely (schema, error handling if the tool call fails).

### Task 1.2 (bonus): Swap a pipeline component

التقدير: 5 نقطة

Show the same agent working with a different STT or TTS provider than your first implementation (or explain precisely, with code snippets, what would change if you swapped providers). This tests whether your design decouples the pipeline from any one vendor.

## Section 2 — LangChain

### Task 2.1: Build a small RAG pipeline

التقدير: 20 نقطة

Build a retrieval-augmented generation pipeline over a small document set (we'll provide 3–5 short PDFs/markdown files, or you may use your own domain docs — state your choice):

- Chunk and embed the documents into a vector store (FAISS, Chroma, or similar).
- Build a LangChain (or LangGraph) chain that retrieves relevant chunks and answers a user question with citations back to the source chunk.
- Handle the "no relevant context found" case explicitly (don't let the model hallucinate an answer).
- Provide 3 example questions and the actual answers your pipeline produced.

Write-up (half page): what would you change about chunking strategy or retrieval (e.g. hybrid search, re-ranking) if answer quality on longer documents were poor?

---

## Section 3 — Quantization

### Task 3.1: Quantize a model and measure the trade-off

التقدير: 20 نقطة

Take an open-weight LLM you can run locally or on free-tier compute (e.g. a small model such as Llama-3.2-1B/3B, Qwen2.5-1.5B, or Mistral-7B if resources allow):

- Run it once at full precision (fp16/bf16) and once quantized, using at least one real technique (e.g. bitsandbytes 4-bit/NF4 via transformers, or a GGUF build via llama.cpp).
- Measure and report: memory footprint (VRAM/RAM), tokens/sec throughput, and qualitative output quality on 5 fixed prompts (same prompts, both versions).
- Summarize the trade-off in a short table: precision vs. size vs. speed vs. quality.

Write-up (half page): when would you pick GPTQ/AWQ over bitsandbytes, or GGUF over both, for a production deployment? Answer from your own experience or reasoning, not just documentation.

---

## Section 4 — Model Deployment

### Task 4.1: Serve the model behind an API

التقدير: 25 نقطة

Take the model from Section 3 (or any LLM) and deploy it as a working service:

- Wrap it behind a REST API (FastAPI is fine) or an inference server (vLLM, TGI) — your choice, but justify it.
- Containerize it with a working Dockerfile (docker build && docker run should work end-to-end).
- Support streaming responses (token-by-token or chunked) for at least one endpoint.
- Include a basic load/latency test: e.g. time-to-first-token and total latency for 10 concurrent requests, with results reported (a simple script or curl/locust output is fine).

Write-up (half page): how would this change if you needed to serve 50 concurrent users in production — what would you add (batching, autoscaling, caching, queueing)?

---

## Submission Checklist

- Code for all 4 sections (Section 1.2 bonus is optional) in a single repo/zip, organized by folder per section.
- A top-level README with setup/run instructions — we should be able to run each section within 10 minutes of cloning.
- Each section's half-page write-up included (in the README or a separate NOTES.md).
- Any assumptions, shortcuts, or known limitations clearly stated — honesty about trade-offs is scored positively.

## Scoring Guide (for internal use)

| Section               | Points        | What we're checking                                                               |
|-----------------------|---------------|-----------------------------------------------------------------------------------|
| 1. LiveKit Agents     | 20 (+5 bonus) | Real-time pipeline understanding, tool-calling correctness, async handling        |
| 2. LangChain / RAG    | 20            | Retrieval quality, hallucination guardrails, chain design                         |
| 3. Quantization       | 20            | Hands-on experience, correct trade-off reasoning, real measurements (not guesses) |
| 4. Model Deployment   | 25            | Production-mindedness: containerization, streaming, latency awareness             |
| Overall communication | 15            | README clarity, write-up quality, honesty about limitations                       |

*Total: 100 points (+5 bonus). Suggested bar for on-site technical interview invite: 65+.*